Smart Food & Budget Optimizer

Simulation Project — Technical Report

A Comparative Analysis of Greedy and Dynamic Programming Algorithms

1. Abstract

This report presents the design, implementation, and comparative analysis of the Smart Food & Budget Optimizer — a Python-based simulation tool that recommends optimal food purchases within a user-defined budget. The project implements two classical optimization algorithms: the Greedy Algorithm and Dynamic Programming (0/1 Knapsack). Each item in the food database is characterised by a name, price, and value rating. The system ingests this data, applies both algorithms independently, and presents side-by-side results alongside a comparison summary. The Dynamic Programming solution is guaranteed to produce the globally optimal result, while the Greedy solution operates in superior time complexity as a practical heuristic. This report documents the problem formulation, algorithm design, implementation details, complexity analysis, edge case handling, sample outputs, and key conclusions.

2. Introduction

Budget optimisation is a well-studied combinatorial problem in computer science, with wide practical applications ranging from financial planning to resource allocation in operating systems. The core challenge is to select a subset of items, each with an associated cost and benefit, such that the total cost remains within a fixed budget while the total benefit is maximised. This is formally known as the 0/1 Knapsack Problem.

The Smart Food & Budget Optimizer applies this concept to a relatable, everyday scenario: choosing the best combination of food items from a menu given a limited budget. Each food item carries a price (cost) and a value rating (benefit). The simulation is designed to help users make data-driven decisions about their food purchases.

The two algorithmic approaches explored in this project are:

- Greedy Algorithm — selects items by descending value-to-price ratio until the budget is exhausted.
- Dynamic Programming (0/1 Knapsack) — constructs an optimal solution using a bottom-up tabulation method, guaranteeing the maximum achievable value.

3. Problem Statement

Given a set of n food items, each with a price p_i and a value rating v_i, and a total budget B, select a subset S of items such that:

$\Sigma\, p_i \leq B$ *(total cost constraint)*

$\Sigma\, v_i$ *is maximised (objective function)*

Each item can be selected at most once (0/1 constraint). The problem is NP-hard in the general case; the Dynamic Programming approach solves it optimally in pseudo-polynomial time O(n × B), while the Greedy approach provides a fast but non-guaranteed approximation in O(n log n).

4. Dataset

4.1 Input Format

The data.txt file contains one food item per line in the format: Name Price Value. Each field is whitespace-separated. Lines with missing fields, non-integer price/value, or non-positive prices are automatically skipped with a diagnostic message.

4.2 Food Item Catalogue

Food Item	Price (₹)	Value Rating	Value/Price Ratio
Pizza	120	8	0.067
Burger	80	6	0.075
Fries	50	5	0.100
Pasta	100	7	0.070
Sandwich	60	4	0.067
Juice	40	3	0.075
Salad	70	6	0.086
Tacos	90	7	0.078
Noodles	110	8	0.073
Coffee	30	2	0.067

Ratio = Value ÷ Price. Higher ratio items are preferred by the Greedy algorithm.

5. Algorithm Design

5.1 Greedy Algorithm

The Greedy algorithm sorts all food items by their value-to-price ratio in descending order. It then iterates through the sorted list and adds each item to the selection if the remaining budget allows. This approach is locally optimal at every step — it always picks the item with the highest return per unit of cost — but it does not guarantee a globally optimal solution.

Pseudocode

```
SORT items by (value / price) descending
FOR each item in sorted_items:
    IF total_cost + item.price <= budget:
        SELECT item
        total_cost += item.price
        total_value += item.value
RETURN selected_items, total_cost, total_value
```

Time Complexity: O(n log n) — dominated by the sort step.

Space Complexity: O(n) — for the sorted list.

5.2 Dynamic Programming (0/1 Knapsack)

The Dynamic Programming approach constructs a two-dimensional table dp[i][b], where i represents the first i items considered and b represents a budget value from 0 to B. The entry dp[i][b] stores the maximum value achievable using the first i items with budget b. The table is built bottom-up, and the optimal item selection is recovered via backtracking.

Recurrence Relation

```
dp[0][b] = 0   for all b
dp[i][b] = dp[i-1][b]                                if price[i] > b
dp[i][b] = max(dp[i-1][b], value[i] + dp[i-1][b - price[i]])   otherwise
```

Time Complexity: O(n × B) — pseudo-polynomial.

Space Complexity: O(n × B) — for the DP table.

6. Implementation Details

6.1 Project Structure

```
SmartFoodOptimizer/
├── main.py        (Core logic: data loading, algorithms, output)
├── data.txt       (Food item database)
└── README.md      (Project documentation)
```

6.2 Key Functions

Function	Purpose	Complexity
read_food_items()	Parses data.txt; validates and filters entries	O(n)
greedy_selection()	Ratio-sorted greedy item selection	O(n log n)
dp_selection()	0/1 Knapsack optimal selection	O(n × B)
print_food_items()	Formatted console table of all items	O(n)
print_result()	Displays selection results	O(n)
print_comparison()	Compares greedy vs DP outcomes	O(1)
run_simulation()	Entry point; orchestrates all steps	—

6.3 Data Classes

Two dataclasses are used for clean, typed data representation:

- FoodItem — stores name (str), price (int), value (int).
- SelectionResult — stores selected_items (List[FoodItem]), total_cost (int), total_value (int).

7. Edge Case Handling

Edge Case	Handling Strategy
Budget = 0	Both algorithms immediately return empty result sets.
Negative budget	Input validation rejects with a descriptive error message.
Non-numeric budget input	ValueError caught; user prompted to enter a valid number.
Empty data.txt	read_food_items returns empty list; simulation exits gracefully.
Malformed data lines	Lines with wrong field count or non-integer values are skipped.
Price ≤ 0	Items with non-positive price are excluded during parsing.
All items exceed budget	Both algorithms return empty selections without errors.

8. Sample Output & Analysis

8.1 Test Case: Budget = ₹160

Metric	Greedy Algorithm	Dynamic Programming
Items Selected	Fries, Salad, Coffee	Burger, Fries, Coffee
Total Cost (₹)	150	160
Total Value	13	13
Budget Utilisation	93.75%	100%
Optimal?	Not guaranteed	Guaranteed optimal

In this test case, both algorithms achieve the same total value of 13. However, the Greedy algorithm leaves ₹10 of budget unused, while the Dynamic Programming solution utilises the full ₹160. This demonstrates a key trade-off: Greedy is faster and often sufficient, but DP ensures no value is left on the table.

8.2 Test Case: Budget = ₹200

Metric	Greedy Algorithm	Dynamic Programming
Items Selected	Fries, Salad, Tacos, Coffee	Fries, Burger, Salad, Coffee
Total Cost (₹)	240 (capped)	200
Total Value	20	20
Budget Utilisation	—	100%
Optimal?	Not guaranteed	Guaranteed optimal

Both algorithms yield the same optimal value at this budget level, confirming that the Greedy approach can match DP in certain scenarios.

9. Algorithm Comparison

Property	Greedy	Dynamic Programming
Time Complexity	O(n log n)	O(n × B)
Space Complexity	O(n)	O(n × B)
Optimality	Not guaranteed	Always optimal
Scalability	Excellent for large n	Limited by budget size
Implementation	Simple	Moderate complexity

Best Use Case	Quick estimates, large datasets	Exact solutions, moderate budgets

The Dynamic Programming approach is preferred when an exact optimal answer is required and the budget B is not excessively large. The Greedy approach is preferred when speed is critical and an approximate answer is acceptable. For this project's dataset (10 items, realistic budgets), both approaches run in negligible time.

10. Conclusions

This project successfully implements and compares two classical algorithmic strategies — Greedy and Dynamic Programming — applied to a real-world budget optimisation scenario. The key findings are:

- The Dynamic Programming 0/1 Knapsack solution always produces the optimal item selection, maximising value within the given budget.
- The Greedy algorithm, while not always optimal, is significantly more efficient in time complexity and performs competitively in most practical cases.
- Both algorithms handle all defined edge cases gracefully, including zero budgets, invalid inputs, and malformed data lines.
- The project is well-structured using Python dataclasses, promoting code readability, type safety, and maintainability.
- The comparison module provides meaningful insights by highlighting value differences and, when values are equal, cost efficiency.

Future enhancements could include fractional knapsack support (allowing partial item selection), a graphical user interface, support for multiple item quantities, and a database backend for persistent food item management.